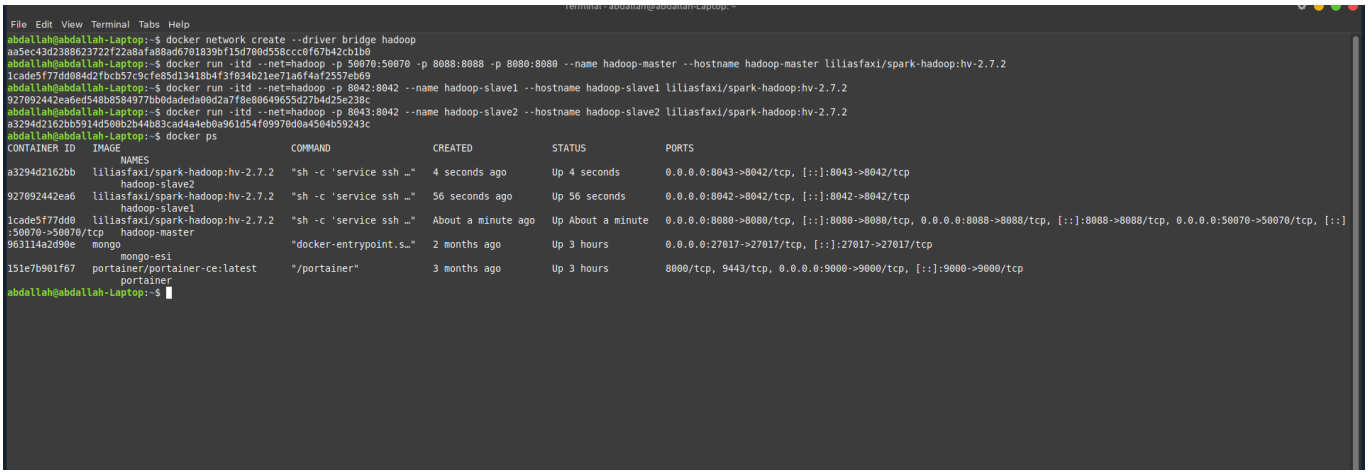
for the creation of the two slave and the master nodes



```
File  Edit  View  Terminal  Tabs  Help
abdallah@abdallah-Laptop:~$ docker network create --driver bridge hadoop
aa5ec43d2388623722f22a8afa88ad6701839bf15d700d558ccc0f67b42cb1b0
abdallah@abdallah-Laptop:~$ docker run -itd --net=hadoop -p 50070:50070 -p 8088:8088 -p 8080:8080 --name hadoop-master --hostname hadoop-master liliasfaxi/spark-hadoop:hv-2.7.2
1cade5f77dd0844d2fbcb57c9cfe85d13418b4f3f034b21ee71a6f4af2557eb69
abdallah@abdallah-Laptop:~$ docker run -itd --net=hadoop -p 8042:8042 --name hadoop-slave1 --hostname hadoop-slave1 liliasfaxi/spark-hadoop:hv-2.7.2
927092442ea6ed548b8584977bb0dadeda00d2a7f8e80649655d27b4d25e238c
abdallah@abdallah-Laptop:~$ docker run -itd --net=hadoop -p 8043:8042 --name hadoop-slave2 --hostname hadoop-slave2 liliasfaxi/spark-hadoop:hv-2.7.2
a3294d2162bb5914d500b2b44b83cad4a4eb0a961d54f09970d0a4504b59243c
abdallah@abdallah-Laptop:~$ docker ps
CONTAINER ID   IMAGE                             COMMAND                  CREATED              STATUS              PORTS
                    NAMES
a3294d2162bb   liliasfaxi/spark-hadoop:hv-2.7.2   "sh -c 'service ssh …"   4 seconds ago        Up 4 seconds        0.0.0.0:8043->8042/tcp, [::]:8043->8042/tcp
                    hadoop-slave2
927092442ea6   liliasfaxi/spark-hadoop:hv-2.7.2   "sh -c 'service ssh …"   56 seconds ago       Up 56 seconds       0.0.0.0:8042->8042/tcp, [::]:8042->8042/tcp
                    hadoop-slave1
1cade5f77dd0   liliasfaxi/spark-hadoop:hv-2.7.2   "sh -c 'service ssh …"   About a minute ago   Up About a minute   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp, 0.0.0.0:50070->50070/tcp, [::]
:50070->50070/tcp   hadoop-master
963114a2d90e   mongo                             "docker-entrypoint.s…"   2 months ago         Up 3 hours          0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
                    mongo-esi
151e7b901f67   portainer/portainer-ce:latest     "/portainer"             3 months ago         Up 3 hours          8000/tcp, 9443/tcp, 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
                    portainer
abdallah@abdallah-Laptop:~$
```

Use the following command to access the master node:

```
docker start hadoop-master hadoop-slave1 hadoop-slave2

docker exec -it hadoop-master bash
```

then

```
root@hadoop-master:~# ./start-hadoop.sh
```

- **for the "Getting Started with Hadoop" section :**

```
Terminal - root@hadoop-master: ~
File  Edit  View  Terminal  Tabs  Help
root@hadoop-master:~# hadoop fs -ls
Found 1 items
drwxr-xr-x   - root supergroup          0 2026-02-02 22:19 input
root@hadoop-master:~# hadoop fs -mkdir -p input
```

```
root@hadoop-master:~# hadoop fs -put purchases.txt input/
root@hadoop-master:~# hadoop fs -ls input/
Found 1 items
-rw-r--r--   2 root supergroup  211312924 2026-02-02 22:23 input/purchases.txt
root@hadoop-master:~# hadoop fs -tail input/purchases.txt
31      17:59   Norfolk Toys    164.34  MasterCard
2012-12-31      17:59   Chula Vista     Music   380.67  Visa
2012-12-31      17:59   Hialeah Toys    115.21  MasterCard
2012-12-31      17:59   Indianapolis    Men's Clothing  158.28  MasterCard
2012-12-31      17:59   Norfolk Garden  414.09  MasterCard
2012-12-31      17:59   Baltimore       DVDs    467.3   Visa
2012-12-31      17:59   Santa Ana       Video Games     144.73  Visa
2012-12-31      17:59   Gilbert Consumer Electronics    354.66  Discover
2012-12-31      17:59   Memphis Sporting Goods  124.79  Amex
2012-12-31      17:59   Chicago Men's Clothing  386.54  MasterCard
2012-12-31      17:59   Birmingham      CDs     118.04  Cash
2012-12-31      17:59   Las Vegas       Health and Beauty       420.46  Amex
2012-12-31      17:59   Wichita Toys    383.9   Cash
2012-12-31      17:59   Tucson  Pet Supplies    268.39  MasterCard
2012-12-31      17:59   Glendale        Women's Clothing        68.05   Amex
2012-12-31      17:59   Albuquerque     Toys    345.7   MasterCard
2012-12-31      17:59   Rochester       DVDs    399.57  Amex
2012-12-31      17:59   Greensboro      Baby    277.27  Discover
2012-12-31      17:59   Arlington       Women's Clothing        134.95  MasterCard
2012-12-31      17:59   Corpus Christi  DVDs    441.61  Discover
root@hadoop-master:~#
```

- **commend to run the Run MapReduce**

```
# Delete previous output
hadoop fs -rm -r output_final

# Run the job
hadoop jar wordcount.jar tp1.WordCount input output_final
```

- **This what i get after Run MapReduce on the Cluster**

```
Terminal - root@hadoop-master: ~
File  Edit  View  Terminal  Tabs  Help
root@hadoop-master:~# hadoop fs -ls output_final
Found 2 items
-rw-r--r--   2 root supergroup          0 2026-02-02 23:03 output_final/_SUCCESS
-rw-r--r--   2 root supergroup     499048 2026-02-02 23:03 output_final/part-r-00000
root@hadoop-master:~# hadoop fs -cat output_final/part-r-00000 | grep -E "Miami|Visa|Cash|New York"
Cash    828770
Miami   39935
Visa    827221
root@hadoop-master:~# hadoop fs -cat output_final/part-r-00000 | grep -E "New|York"
New     80278
Newark  40577
York    40364
root@hadoop-master:~#
```

- to see the full result use :

```
    hadoop fs -cat output_final/part-r-00000
```

EXO 2 : Word count

- **the problem :** solve the WordCount problem

for the second part we have two create this two python codes

- **Create Mapper:**
- The Organizer: reads the input line-by-line and "maps" it into organized pairs of (Key, Value).

```
import sys

# standard input
for line in sys.stdin:
    line = line.strip()
    words = line.split() # line to words


    for word in wordthe problem :s:
        print('%s\t%s' % (word, 1))
```

- **Shuffle & Sort**: Hadoop does this automatically. It takes all the same Keys and groups them together next to each other.
- **Create Reducer:**
- *The Calculator :* Takes a specific Key and a list of Values, and produces a final result.

```
import sys

current_word = None
current_count = 0
word = None

# Read input stdin : allowing the exact same code to process data piped
from a local file or a massive Hadoop cluster without modification
for line in sys.stdin:
    line = line.strip()

    # Parse the input we got from mapper.py
    try:
        word, count = line.split('\t', 1)
        count = int(count)
    except ValueError:
        continue  # bad lines
```

```
    if current_word == word:
        current_count += count # Add to the total
    else:
        # New word => we finished the previous word. print the total.
        if current_word:

            print('%s\t%s' % (current_word, current_count))
        current_count = count
        current_word = word

# output very last word
if current_word == word:
    print('%s\t%s' % (current_word, current_count))
```

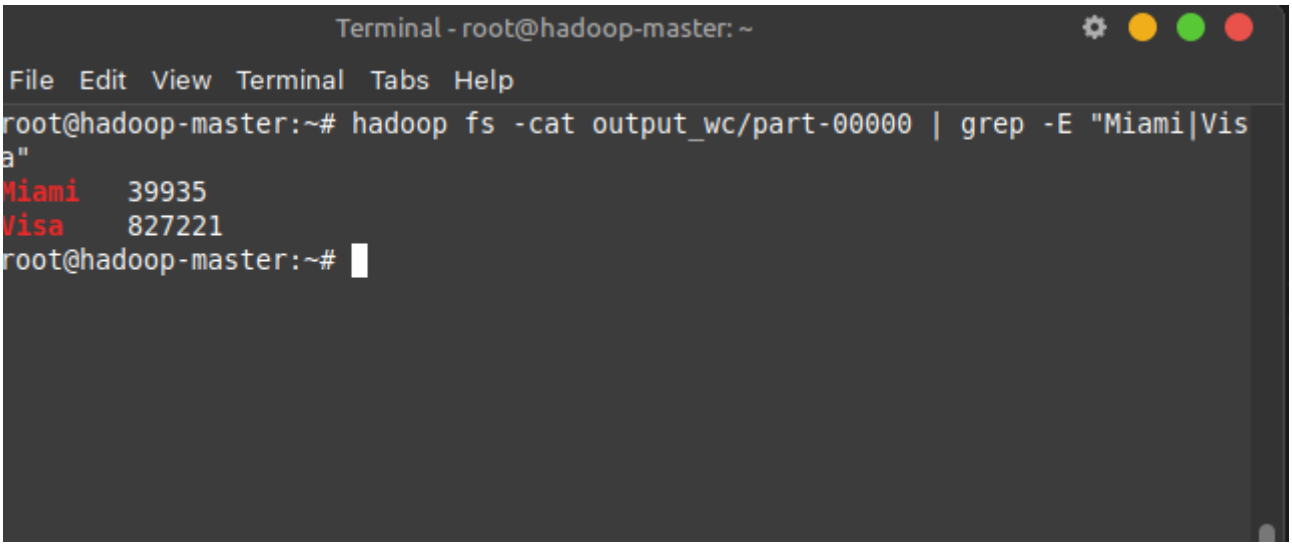
- and after run it with :

```
hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-
2.7.2.jar \
    -file mapper_wc.py    -mapper mapper_wc.py \
    -file reducer_wc.py   -reducer reducer_wc.py \
    -input input/purchases.txt \
    -output output_wc`
```

we will get Output directory: output_python that you can see using

```
hadoop fs -cat output_wc/part-00000 | grep -E "Miami|Visa"
```

the output:



EXO 3

the problem:

calculate the total sales per store

The code :

- **The Mapper**:

```
import sys

for line in sys.stdin:
    line = line.strip()
    # Split  line into columns
    data = line.split('\t')

    if len(data) == 6:
        date, time, store, product, cost, payment = data

        # Output: StoreName  Cost
        print('%s\t%s' % (store, cost))
```

- **The Shuffle**:Hadoop automatically Sorts and Groups the data
- **The Reducer**:

```
import sys

last_store = None
total = 0.0       Output:

for line in sys.stdin:
    line = line.strip()

    #line into Store and Cost
    try:
        store, cost = line.split('\t', 1)
        cost = float(cost)

    except ValueError:
        continue  # Skip bad lines


    if last_store == store:
        # same store => add
        total += cost
    else:
        # The store changed

        # Print the total of OLD store
        if last_store:
            print('%s\t%s' % (last_store, total))

        # Reset for NEW store
```

```
        last_store = store
        total = cost


if last_store:
    print('%s\t%s' % (last_store, total))
```

See the results

```
hadoop fs -cat output_sales/part-00000 | head
```

the result

```
root@hadoop-master:~# hadoop fs -cat output_sales/part-00000 | head
Albuquerque     10052311.419999966
Anaheim 10076416.359999955
Anchorage       9933500.400000004
Arlington       10072207.97000006
Atlanta 9997146.700000083
Aurora  9992970.91999996
Austin  10057158.900000017
Bakersfield     10031208.920000002
Baltimore       10096521.450000018
Baton Rouge     10131273.229999963
```

EXO 4

the problem :

sorted list of dates in the format yyyy-mm-dd for each location where the government in Nigeria regained territory

The code:

the Mapper: act as filter

```
import sys
import csv
from datetime import datetime


reader = csv.reader(sys.stdin)

for row in reader:
    # skip empty lines /the header row
    if not row or len(row) < 4:
        continue

    # what we need
```

```
    location = row[0]
    date_str = row[1]
    event_type = row[3]

    # check if 'regains' is in the event type
    if 'regains' in event_type.lower():
        try:
            # m/d/yy --> yyyy-mm-dd
            dt = datetime.strptime(date_str, "%m/%d/%y")
            formatted_date = dt.strftime("%Y-%m-%d")

            print('%s\t%s' % (location, formatted_date))
        except ValueError:

            continue
```

the Reducer:

```
import sys

current_location = None
dates = []

for line in sys.stdin:
    line = line.strip()
    try:
        location, date = line.split('\t', 1)
    except ValueError:
        continue

    # location is the same =>  collect the date
    if current_location == location:
        dates.append(date)
    else:
        # New locationregains
        if current_location:
            dates.sort()
            print('%s\t%s' % (current_location, dates))

        current_location = location
        dates = [date]


if current_location:
    dates.sort()
    print('%s\t%s' % (current_location, dates))
```

Run the Job

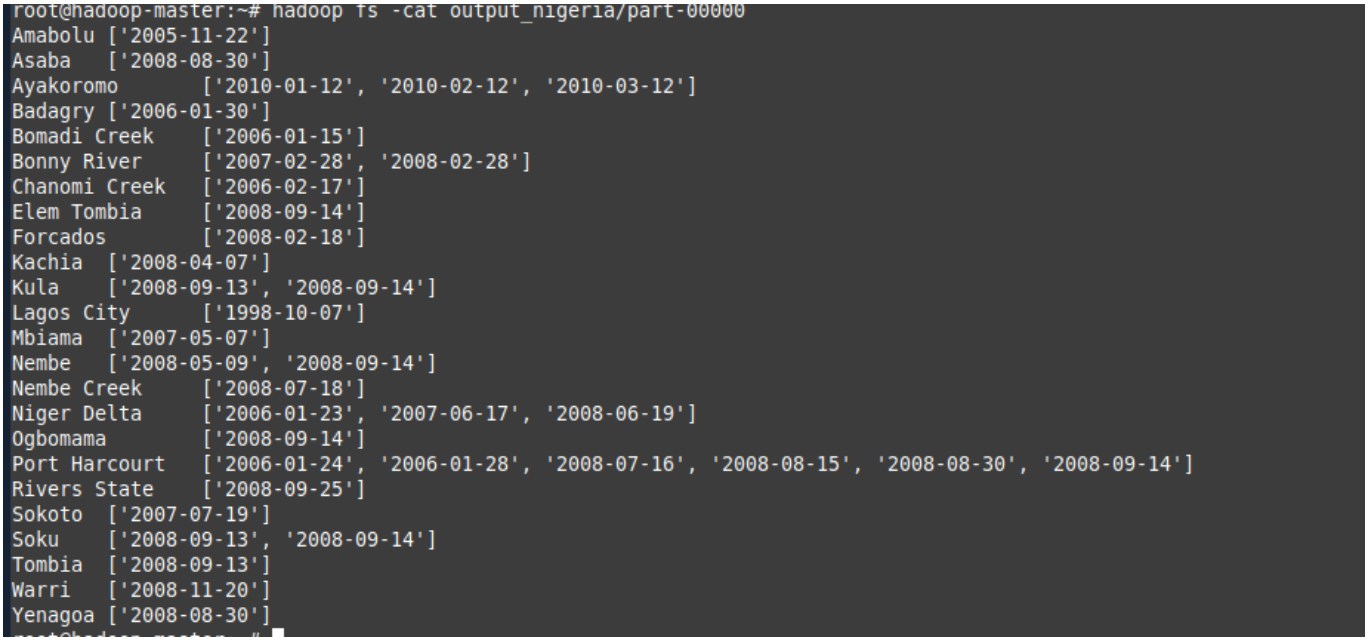
```
JAR=/usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-2.7.2.jar

hadoop jar $JAR \
    -file mapper_nigeria.py    -mapper mapper_nigeria.py \
    -file reducer_nigeria.py   -reducer reducer_nigeria.py \
    -input input/Nigeria.csv \
    -output output_nigeria
```

View the Results

```
hadoop fs -cat output_nigeria/part-00000
```

the Result :



```
root@hadoop-master:~# hadoop fs -cat output_nigeria/part-00000
Amabolu ['2005-11-22']
Asaba   ['2008-08-30']
Ayakoromo       ['2010-01-12', '2010-02-12', '2010-03-12']
Badagry ['2006-01-30']
Bomadi Creek    ['2006-01-15']
Bonny River     ['2007-02-28', '2008-02-28']
Chanomi Creek   ['2006-02-17']
Elem Tombia     ['2008-09-14']
Forcados        ['2008-02-18']
Kachia  ['2008-04-07']
Kula    ['2008-09-13', '2008-09-14']
Lagos City      ['1998-10-07']
Mbiama  ['2007-05-07']
Nembe   ['2008-05-09', '2008-09-14']
Nembe Creek     ['2008-07-18']
Niger Delta     ['2006-01-23', '2007-06-17', '2008-06-19']
Ogbomama        ['2008-09-14']
Port Harcourt   ['2006-01-24', '2006-01-28', '2008-07-16', '2008-08-15', '2008-08-30', '2008-09-14']
Rivers State    ['2008-09-25']
Sokoto  ['2007-07-19']
Soku    ['2008-09-13', '2008-09-14']
Tombia  ['2008-09-13']
Warri   ['2008-11-20']
Yenagoa ['2008-08-30']
```

EXO 5

the problem :

- find groups of words that are Anagrams of each other.

Create Mapper : sort letters alphabetically

```
import sys

for line in sys.stdin:
    words = line.strip().split()
    for word in words:
```

```
        # Key = sorted letters (e.g., "melon" -> "elmno")
        sorted_key = "".join(sorted(word))
        print('%s\t%s' % (sorted_key, word))
```

Create Reducer

```
import sys

last_key = None
anagrams = []

for line in sys.stdin:
    try:
        key, word = line.strip().split('\t', 1)
    except: continue

    if last_key == key:
        anagrams.append(word)
    else:
        if last_key:
            print('%s\t%s' % (last_key, anagrams))
        last_key = key
        anagrams = [word]

if last_key:
    print('%s\t%s' % (last_key, anagrams))
```

the result

```
root@hadoop-master:~# hadoop fs -cat output_anagram/part-00000
abber   ['barbe']
abefil  ['faible', 'fiable']
abefl   ['fable']
aberr   ['arbre', 'barre']
deeinrv ['deviner', 'devenir']
deenrv  ['vendre']
elmno   ['lemon', 'melon']
```